

Chapter 11 AVL Tree

AVL Trees

Prompt

What is an AVL tree?

Answer

- An **AVL tree** is a **self-balancing binary search tree** (BST)
- Named after **Adelson-Velskii and Landis** (AVL)
- It satisfies the **height-balance property**:

For every node p in the tree, the heights of its **left and right children differ by at most 1**

- **Leaves** have height = 1
- **Null children** have height = 0

Prompt

What is the height-balance property of an AVL tree?

Answer

Let $h(\text{left})$ and $h(\text{right})$ be heights of left and right subtrees of a node p .

Condition:

$$|h(\text{left}) - h(\text{right})| \leq 1$$

- If this holds for all nodes $\rightarrow$ tree is AVL
- Consequence: **subtrees of an AVL tree are also AVL trees**

Prompt

How does the height of an AVL tree relate to the number of nodes?

Answer

- Let $n(h)$ = minimum number of nodes in an AVL tree of height h
- Recurrence formula:

$$n(1) = 1, n(2) = 2$$

$$n(h) = 1 + n(h-1) + n(h-2), h \geq 3$$

- Reason: Minimum AVL tree has:
 - One subtree of height $h-1$
 - One subtree of height $h-2$
 - Root node adds 1
- **Observation:** $n(h)$ grows exponentially $\rightarrow$ height grows logarithmically

Step 1: Exponential Growth

Step 1: Exponential Growth

$$n(h) > 2 \cdot n(h-2)$$

- Doubling every 2 levels $\rightarrow$ exponential growth
- Applying repeatedly:

$$n(h) > 2^i \cdot n(h-2i)$$

- Choose $i = \frac{h}{2} - 1 \rightarrow$ base case $n(1) = 1$

$$n(h) > 2^{h/2-1} \cdot 1 = 2^{h/2-1}$$

Step 2: Solve for Height

Step 2: Solve for Height

$$\log_2(n(h)) > \frac{h}{2} - 1$$

$$h < 2 \cdot \log_2(n(h)) + 2$$

✓ **Conclusion:** Height of AVL tree = $O(\log n)$

- This ensures **search, insertion, and deletion** run in **$O(\log n)$ time**

Prompt

Can you show an example AVL tree with heights?

Answer

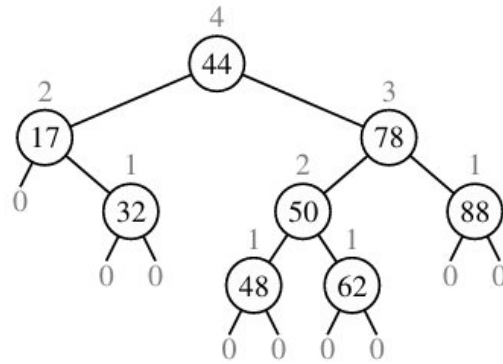


Figure 11.11: An example of an AVL tree. The keys of the items are shown inside the nodes, and the heights of the nodes are shown above the nodes (with empty subtrees having height 0).

- Numbers in **parentheses** = height of subtree
- Balance factor (left height - right height) ≤ 1 at every node
- **Check balance factor:**
 - Node 44: left 2, right 3 $\rightarrow |2-3|=1$ ✓
 - Node 78: left 1, right 1 $\rightarrow |1-1|=0$ ✓
 - Node 17: left 1, right 1 $\rightarrow |1-1|=0$ ✓

AVL Tree Insertion

Prompt

What happens when a new node is inserted into an AVL tree?

Answer

- Insert the new key like a normal **BST insertion** → it becomes a **leaf** p
- The **AVL property** (balance factor ≤ 1) may be **violated**
- Only **ancestors of the new node** p can become unbalanced
- Solution: **search up from p to find first unbalanced ancestor z** , then rebalance

Prompt

How do we find the unbalanced node after insertion?

Answer

1. Start at p → traverse **up toward the root**
2. Let z = first **unbalanced ancestor** of p
3. Let y = child of z with **higher height** (must be ancestor of p)
4. Let x = child of y with **higher height** (could be p itself)
 - Only **subtree rooted at z** needs restructuring

Prompt

How do we restore balance using trinode restructuring?

Answer

- Subtree rooted at z may be **left-heavy or right-heavy**
- Perform **trinode restructuring** with nodes (x, y, z) :

Steps:

Identify nodes x, y, z :

- z = first unbalanced ancestor
- y = child of z with greater height
- x = child of y with greater height

Apply one of **four rotation cases**:

- **LL (Left-Left)** → single right rotation
- **RR (Right-Right)** → single left rotation
- **LR (Left-Right)** → left rotation on y , right rotation on z

- **RL (Right-Left)** → right rotation on y , left rotation on z

Heights of nodes (x , y , z) are **updated** → balance restored

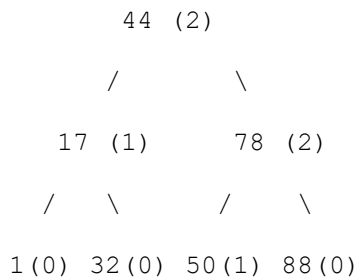
Subtree height after restructuring = height before insertion

Prompt

Show an example of inserting key 54 into an AVL tree and restoring balance.

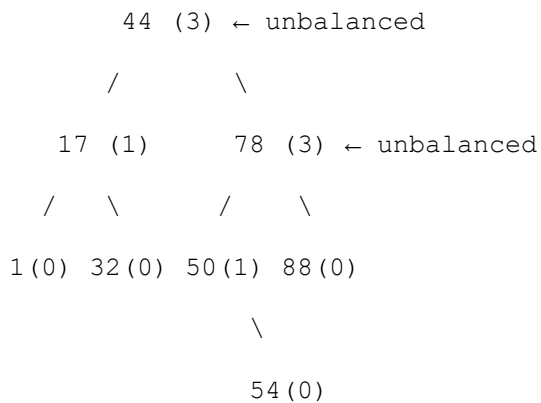
Answer

Step 1: Original AVL Tree (before insertion)



- Numbers in parentheses = height
- Insert **54** → becomes a **leaf** in subtree T3 (right child of 50)

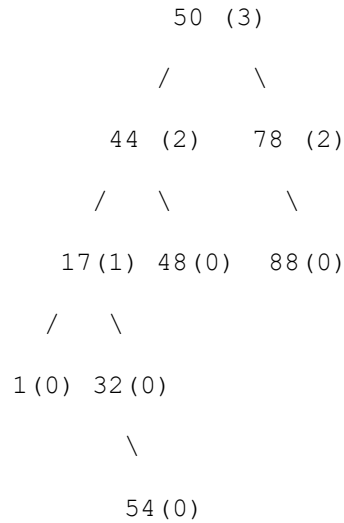
Step 2: After insertion (unbalanced nodes)



- **Node 78** and **node 44** are now unbalanced
- Need **trinode restructuring** with nodes:
 - $z = 44, y = 78, x = 54$

Step 3: Trinode Restructuring

- Identify rotation case: **Left-Right or Right-Left**
- Rotate to restore balance → new subtree:



- **Height of subtree = same as before insertion**
- Balance restored

AVL Tree Deletion

Prompt

What happens when a node is deleted from an AVL tree?

Answer

- Delete the node **like a regular BST deletion**:
 - If the node has **0 children** → remove leaf
 - If the node has **1 child** → promote the child to the deleted node's position
- **AVL balance property may be violated**
- Only nodes on the path from the **parent of deleted node to the root** can become unbalanced

Prompt

How do we find the node that became unbalanced after deletion?

Answer

- Start at p = **parent of deleted node**
- Traverse **up toward the root**
- Let z = first **unbalanced ancestor** of p
- Let y = **child of z with larger height**
- Let x = child of y chosen as:
 - Taller child if heights differ
 - If equal heights $\rightarrow$ pick the child **on same side as y** (left if y is left child of z , right if y is right child of z)
- Perform **trinode restructuring on x** $\rightarrow$ restores local balance

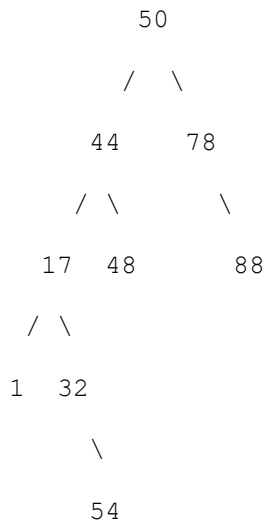
Prompt

Why do we need to check ancestors after a deletion?

Answer

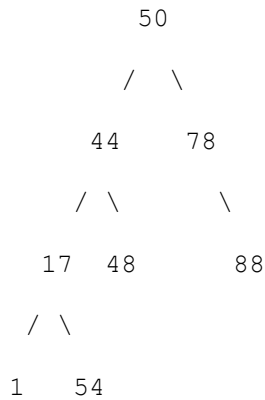
- Unlike insertion, **deletion can decrease the height** of a subtree
- Height decrease may **propagate imbalance upward**
- After restoring balance at z , continue **walking up to the root**, performing trinode restructuring if needed
- Since tree height = $O(\log n)$, **$O(\log n)$ rotations** are sufficient to restore global balance

Step 1: Original AVL Tree Before Deletion



- **Delete 32** (leaf node)

Step 2: After Deletion (possible imbalance)

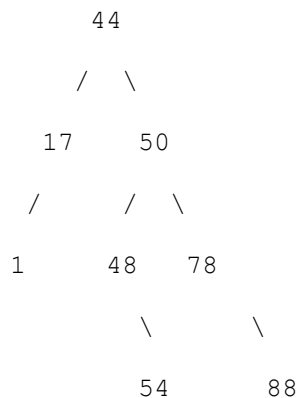


- Check balance factors from parent of deleted node (17) up to root
- Suppose root (50) becomes **unbalanced**

Step 3: Identify nodes for trinode restructuring

- z = first unbalanced node = **50**
- y = child of z with larger height = **44**
- x = child of y = **17** (left child, same side as y)
- Apply **restructure(x)** → rotation to restore balance

Step 4: After Trinode Restructuring



- Heights updated
- Balance property restored locally
- Continue checking up to root (if necessary)

Performance of AVL Trees

Prompt

Why is the height of an AVL tree $O(\log n)$?

Answer

- **AVL trees maintain the height-balance property:** for every node, the heights of left and right subtrees differ by at most 1.
- This property **guarantees that the tree is “balanced”**, preventing long paths.
- Let $n(h)$ = minimum number of nodes in AVL tree of height h .
- Recursive formula:
 - $n(h) = 1 + n(h-1) + n(h-2)$
- This grows exponentially $\rightarrow$ height $h = O(\log n)$
- **Implication:** all operations that depend on tree height (search, insert, delete) are logarithmic in worst case

Prompt

How do AVL trees affect the runtime of typical BST operations?

Answer

Operation	Worst-case Time	Explanation
Search / Get (k in T)	$O(\log n)$	Traverses path from root to leaf $\rightarrow$ height = $O(\log n)$
Insertion ($T[k] = v$)	$O(\log n)$	Search + possible rotations along path $\rightarrow O(\log n)$
Deletion ($T.delete(p)$ / $del\ T[k]$)	$O(\log n)$	Search + possible rotations along path $\rightarrow O(\log n)$
Find min / max / first / last	$O(\log n)$	Walk along leftmost or rightmost path
Successor / Predecessor ($before(p)$, $after(p)$)	$O(\log n)$	Go up to find nearest smaller/larger node
Range queries ($find_range(start, stop)$)	$O(s + \log n)$	s = number of items reported
Iteration over all elements ($iter(T)$)	$O(n)$	Must visit every node

Key idea: The **rotations and height updates** during insert/delete only require $O(1)$ time per node along the path $\rightarrow$ total $O(\log n)$

Prompt

What is the “down phase” and “up phase” in AVL operations?

Answer

- **Down phase:** traverse from root to target node (for search, insert, or delete)
 - At each level: $O(1)$ work $\rightarrow$ total $O(\text{height}) = O(\log n)$
- **Up phase:** after insertion/deletion, walk back up to **update heights and perform rotations** if needed
 - Each level: $O(1)$ work (check balance, rotate if needed)
 - Total $O(\log n)$

Total time = **$O(\log n)$** in worst case